

MAANAK: HYBRID RAG ARCHITECTURE

Team Profile: AI-Assisted "Vibe Coding"

Hackathon Sprint: 36 Hours

Focus: Zero Hallucination Compliance

1. THE CORE CONTEXT

The Problem: The Bureau of Indian Standards (BIS) regulates thousands of product certifications. Currently, Micro, Small, and Medium Enterprises (MSMEs) must read through highly complex, dense, multi-column PDFs filled with tabular specifications and legal clauses to find testing requirements. This manual search process is inefficient, linguistically exclusionary, and prone to human error.

The Expected Solution: A Retrieval-Augmented Generation (RAG) chatbot capable of interpreting these complex documents and answering user queries. The system must explicitly cite the official source clause, mathematically eliminate the risk of the AI hallucinating (inventing fake laws), and be accessible to non-technical users.

2. OVERVIEW OF OUR SOLUTION (THE HYBRID APPROACH)

Strategic Execution

We are deploying a pragmatically engineered, monolithic RAG architecture optimized specifically for a 36-hour execution constraint. Instead of falling into the trap of over-engineered, un-deployable microservices, we combine speed with extreme mathematical precision.

To bypass the trap of traditional PDF parsers destroying tabular data, we utilize a **Multimodal Vision Ingestion Pipeline** (GPT-4o-mini). We ensure absolute accuracy by layering a **Hybrid Search (Dense Vector + BM25)** on Pinecone Serverless, followed by a Cross-Encoder Reranker. This pipeline is protected by a cascading API-key failover system to guarantee 100% demo uptime, while maintaining an isolated frontend/backend workflow for parallel team execution.

3. FEATURE SCOPE (MVP VS. ENHANCEMENTS)

Component Category	Phase 1: Minimum Viable Product (MVP)	Phase 2: High-Value Enhancements
Data Ingestion	Static local ingestion. Python script parses PDFs to Markdown via GPT-4o-mini Vision, chunks, and uploads to Pinecone.	(None needed. Vision parsing is robust enough as-is).
Retrieval & RAG Engine	Hybrid Search (Dense + BM25) querying Pinecone. English language input/output only. Text-based source citations.	Cross-Encoder Reranking implemented to guarantee top-3 chunks are mathematically relevant. API Key Cascading Failover.
User Interface (Next.js)	Basic responsive chat interface (Tailwind/shadcn). Markdown rendering for responses and static text links for citations.	Dual-Pane Evidence UI. react-pdf integration that dynamically opens the source PDF and auto-scrolls to the cited page.
Localization & Voice	Text-only, English language processing.	Bhashini API integration. Real-time Audio-to-Text (ASR) and Hindi-to-English translation before hitting the RAG pipeline.

4. PHASES OF THE BUILD (36-HOUR SPRINT)

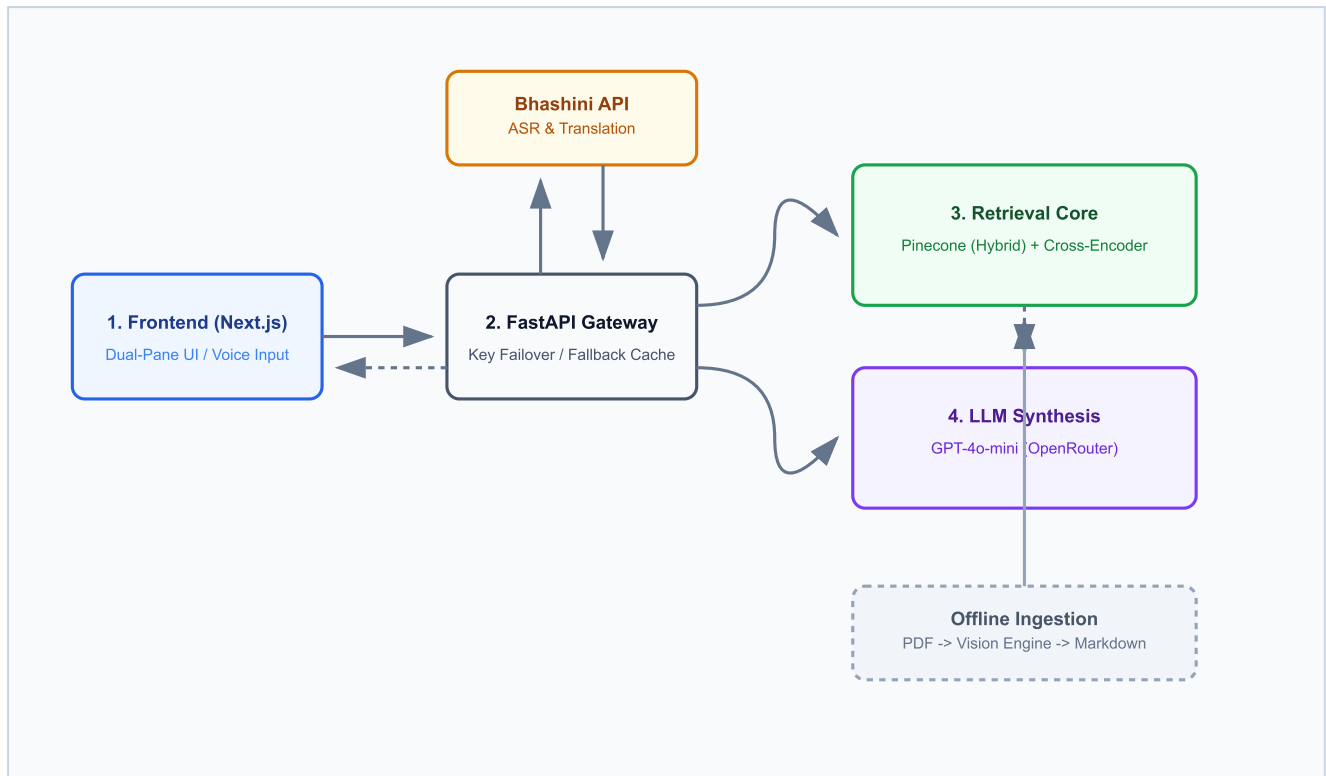
HOURS 0 - 16 Phase 1: The Bulletproof MVP - Agree on JSON API Contracts. - Dev 1 builds Next.js UI using mocked dummy data. - Dev 3 parses PDFs & indexes vectors into Pinecone. - Dev 2 builds FastAPI endpoint & connects LLM. - Dev 4 connects API to Frontend (Integration Point).	HOURS 16 - 28 Phase 2: Standout Enhancements - Implement cascading array-based API key rotation (Failover). - Dev 3 builds standalone Bhashini Localization module. - Dev 1 attempts React-PDF auto-scroll (Timeboxed: 3 hrs).	HOURS 28 - 36 Phase 3: Demo Hardening - Code freeze on backend logic. - Dev 4 injects Hardcoded JSON Cache for the 3 live pitch questions. - UI Polish and exact rehearsal of the presentation flow.
---	---	---

5. TECH STACK & ARCHITECTURAL PIPELINE

The Stack:

- **Frontend:** Next.js (App Router), Tailwind CSS, shadcn/ui.
- **Backend:** Python FastAPI, Uvicorn, LangChain core libraries.
- **Vector Database:** Pinecone Serverless (configured for Hybrid Dense/Sparse Search).
- **LLM Inference:** GPT-4o-mini (via OpenRouter API for redundancy).
- **External APIs:** Bhashini API (Government Localization).

6. WORKFLOW DIAGRAM



7. DIVISION OF LABOR & ISOLATION STRATEGY

To eliminate merge conflicts during concurrent "Vibe Coding," the repository is divided into strict domains. Developers must not edit files outside their assigned directories. The 6-person team is allocated as follows:

Dev 1: Frontend Edge (Next.js)

Scope: Isolated to /frontend directory.

- Builds the chat interface and UI using mock JSON payloads.
- Responsible for Tailwind layouts and shadcn components.
- Phase 2 Task: Implements the react-pdf dual-pane viewer and auto-scroll logic.

Dev 2: Backend Generation (FastAPI)

Scope: Isolated to /backend/main.py.

- Builds the API routes, CORS handling, and OpenRouter LLM connection.
- Tests LLM prompts using hardcoded mock retrieval chunks.
- Phase 2 Task: Engineers the cascading API key rotation to prevent rate-limit crashes.

Dev 3: Data Retrieval & Bhashini

Scope: Isolated to backend scripts (e.g., retriever.py, bhashini.py).

- Engineers the PDF-to-Markdown vision ingestion script.
- Configures Pinecone Serverless and Hybrid Search.
- Phase 2 Task: Builds the standalone Bhashini API wrapper for translation.

Dev 4: DevOps, Evaluation & Integration

Scope: Repository management and glue code.

- Acts as Merge-Master. Resolves any rare Git conflicts.
- Hours 14-16: Plugs Dev 3's live database into Dev 2's FastAPI route.
- Phase 3 Task: Configures the hardcoded live-pitch fallback cache.

Team Support: Product Strategy & Presentation (2 Members)

The remaining 2 team members are strictly prohibited from touching the codebase. Their tasks are:

- **SIH Portal Assets:** Design the mandatory PowerPoint presentation, ensuring it aligns perfectly with the MVP's final feature set.
- **Demo Scripting:** Write the exact 3-question script for the live pitch. Hand these exact questions and answers to Dev 4 to encode into the fallback cache.
- **Video Production:** Record and edit the required demonstration video for the hackathon submission portal while the devs are executing Phase 3 polish.